

MemRL 论文精读与全文解读

MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory

上海交通大学 西安电子科技大学 新加坡国立大学 等

精读文档

March 10, 2026

Contents

1 摘要 (Abstract)	3
2 引言 (Introduction)	3
3 相关工作 (Related Works)	5
3.1 运行时学习 (Runtime Learning)	5
3.2 强化学习 (Reinforcement Learning)	6
3.3 智能体记忆 (Agentic Memory)	6
4 问题形式化 (Problem Formulation)	7
4.1 记忆增强智能体策略 (Memory-Augmented Agent Policy)	8
4.2 非参数强化学习 (Non-Parametric Reinforcement Learning)	9
5 MemRL 方法 (Method)	10
5.1 意图-经验-效用三元组 (The Intent-Experience-Utility Triplet)	11
5.2 从语义召回到价值感知选择 (Two-Phase Retrieval)	12
5.3 基于记忆的非参数强化学习 (Non-Parametric RL on Memory)	13
5.4 理论稳定性分析 (Theoretical Stability Analysis)	13
6 实验 (Experiments)	14
6.1 实验设置 (Experimental Setup)	14
6.2 主要结果 (Main Results)	15
6.2.1 运行时学习结果	15
6.2.2 迁移学习结果	16
6.3 消融实验 (Ablations)	16
6.3.1 运行时 RL 的有效性	16
6.3.2 Q 值权重因子的影响	17
6.3.3 检索范围消融: 跨任务 vs 单任务	17
6.3.4 检索规模敏感性分析	18
6.4 讨论 (Discussion)	18
6.4.1 Q 评论家的预测能力	18
6.4.2 MemRL 的稳定性	19
7 局限与结论 (Limitations and Conclusion)	20

8 影响声明 (Impact Statement)	20
A 附录 A: 理论分析与证明	21
A.1 稳定性分析	21
A.2 有界方差	21
A.3 通过变分推断的收敛性 (GEM 分析)	22
B 附录 B: 扩展分析	22
B.1 遗忘动态分析	22
B.2 MemRL 作为轨迹验证器	23
B.3 任务相似度对记忆效力的影响	23
C 附录 C: 高级能力——迁移与合并	24
C.1 跨模型记忆迁移	24
C.2 多任务记忆合并	24
D 附录 D: 实现细节	24
D.1 模型配置	24
D.2 超参数设置	24
D.3 数据划分	25
E 附录 E: 统一骨干对比实验	25
F 附录 F: 效率分析	26
G 附录 G: 参考文献一览表	27

1 摘要 (Abstract)

原文完整翻译

人类智能的标志性特征在于通过从过往经验中学习来掌握新技能的自我进化能力。然而，当前的AI智能体难以模拟这种自我进化：微调（fine-tuning）计算成本高昂且容易导致灾难性遗忘（catastrophic forgetting），而现有的基于记忆的方法依赖于被动的语义匹配，经常检索到噪声信息。为了应对这些挑战，我们提出了 MemRL，一种通过在情景记忆（episodic memory）上进行强化学习来实现进化的非参数方法。通过将稳定的推理能力与可塑的记忆解耦，MemRL 采用两阶段检索（Two-Phase Retrieval）机制来过滤噪声，并通过环境反馈识别高效用（high-utility）策略。在 HLE、BigCodeBench、ALFWorld 和 Lifelong Agent Bench 上的广泛实验表明，MemRL 显著优于现有最先进的基线方法，证实了 MemRL 有效地调和了稳定性-可塑性困境（stability-plasticity dilemma），实现了无需权重更新的持续运行时改进。代码可在 <https://github.com/MemTensor/MemRL> 获取。

通俗解释

想象你是一个刚入职的员工。人类之所以能快速适应新工作，是因为我们会不断积累经验——成功的经验我们会记住并复用，失败的教训也会帮助我们避免再犯。但是目前的 AI 系统在“从经验中学习”这件事上做得不好。

传统方法有两条路：（1）**微调模型**——相当于“改造大脑”，不仅昂贵（想象做一次脑部手术的费用），还可能让你忘掉以前学会的东西（灾难性遗忘）；（2）**检索增强生成（RAG）**——相当于“查百科全书”，但它只按照“相似度”来找内容，并不知道找到的内容到底“有没有用”。

MemRL 的核心思路是：**不改造大脑，而是升级你的笔记本**。它把大语言模型（LLM）当作固定不变的“大脑”，在外部维护一个会不断进化的“经验记忆库”。通过强化学习给每条记忆打上一个“效用分数”（Q值），下次检索时不仅看“相不相似”，还看“有没有用”——这就像一个老员工不仅记得做过什么，还知道哪些做法真正管用。

核心要点

- **核心问题**：如何让 AI 智能体在部署后持续改进，而不破坏预训练模型的稳定性？
- **解决方案**：MemRL——一种非参数的、基于强化学习的记忆管理框架
- **关键创新**：Intent-Experience-Utility 三元组 + 两阶段检索 + 基于环境反馈的效用更新
- **核心优势**：不需要更新模型权重，避免灾难性遗忘，同时实现持续性能提升

2 引言 (Introduction)

原文完整翻译

人类智能在认知稳定性与情景可塑性之间取得了平衡【Grossberg 2013, McClelland 1995, Kumaran 2016——关于大脑中稳定性-可塑性平衡的经典认知科学文献，说明人类大脑既能保持已有知识又能学习新知识】，这得益于建构性情景模拟（Constructive Episodic Simulation）机制，使人类无需重新布线神经回路即可适应新环境【Schacter 2007, Hassabis 2007, Schacter 2012, Gick 1980——关于人类如何通过回忆过去经历来模拟未来场景的认知科学研究，解释了本文的生物学灵感来源】。尽管当前 AI 智能体具备推理能力【Wei 2022 (Chain-of-thought)、Yao 2022 (ReAct)、Schick 2023

(Toolformer)、Wang 2023 (Voyager)——展示 LLM 推理能力的代表性工作，说明 LLM 可作为智能体的认知基础，但它们难以模拟这种解耦的自我进化。具体而言，微调通过修改权重来内化经验【Ouyang 2022 (InstructGPT)、Stiennon 2020 (RLHF)、Rafailov 2023 (DPO)、Ethayarajh 2024 (KTO)——用人类反馈训练语言模型的代表性方法】，但受限于计算成本和灾难性遗忘【Kirkpatrick 2017 (EWC)、Li 2024、Wu 2024——关于灾难性遗忘问题的研究，说明微调的风险】。相反，检索增强生成 (RAG)【Lewis 2020——RAG 的原始论文，提出了用外部知识库增强生成的范式】提供了一种非参数替代方案，但本质上是被动的，仅按语义相似度检索而非效用【Karpukhin 2020 (DPR)、Gao 2023——关于密集检索和 RAG 综述的工作】；这使得智能体无法有效利用运行时反馈来区分高价值策略和噪声。

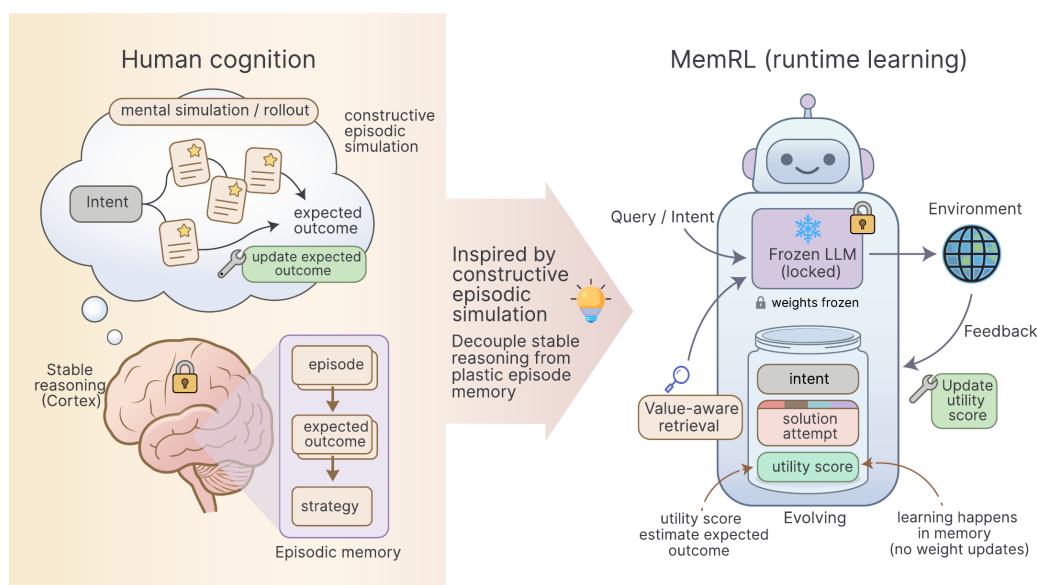


Figure 1: MemRL 的概念框架。展示了从人类认知启发到 MemRL 方法的映射关系。

图 1 详细解读

这张图展示了 MemRL 的概念框架，将人类认知机制与 AI 系统进行了类比：

核心思想：人类大脑将”认知推理能力”（稳定的）与”情景记忆”（可塑的）解耦。MemRL 模仿这一机制：冻结的 LLM 对应稳定的认知能力，外部记忆库对应可塑的情景记忆。

关键对应关系：（1）人类的”类比迁移”对应 MemRL 的 Phase-A 语义检索——根据相似性找到相关经验；（2）人类的”心理预演”对应 Phase-B 基于效用的选择——在候选经验中选择最有用的；（3）人类的”记忆再巩固”对应 Q 值更新——根据结果反馈强化或削弱记忆的效用值。

原文完整翻译（续）

这一局限性凸显了一个关键研究问题：**如何让智能体在部署后持续改进性能，同时不损害其预训练骨干模型的稳定性？**我们的目标是实现一个随着持续使用而进化的智能体，并在部署后快速适应新任务，即运行时持续学习（Runtime Continuous Learning）【Javed 2023, Silver 2025 (经验时代)、Parisi 2019、Wu 2024——关于持续学习和运行时学习的研究，定义了本文所针对的问题设置】，同时保持骨干模型冻结以防止灾难性遗忘【Finn 2017 (MAML)、Wei 2025 (Evo)——关于元学习和智能体进化的工作】。

为了应对这一挑战，受人类建构性模拟认知机制的启发，我们提出了 MemRL，一种通过

显式解耦模型稳定认知推理与动态情景记忆来促进智能体自我进化的方法。图 1 展示了所提出的 MemRL 的概念框架。

借鉴强化学习 (RL) 中试错方式来估计预期经验效用【Sutton & Barto 2018——**强化学习经典教材，提供了 RL 的理论基础**】，我们将冻结 LLM 与外部记忆之间的交互形式化为马尔可夫决策过程 (MDP)【Puterman 2014——**MDP 的经典参考文献**】。与优化骨干模型的传统方法不同，MemRL 在不调整模型权重的情况下优化记忆使用策略。

MemRL 将记忆组织成结构化的意图-经验-效用 (Intent-Experience-Utility) 三元组。这种结构将检索从被动的语义匹配任务转变为主动的决策过程：两阶段检索根据学习到的 Q 值（反映预期效用）而非仅依赖语义相似度来选择经验【Watkins 1992——**Q-learning 的原始论文**】；效用驱动更新通过环境反馈应用蒙特卡洛风格更新来精化这些 Q 值【Metropolis 1949——**蒙特卡洛方法的原始论文**】。

这一闭环循环使智能体能够区分高价值记忆和相似噪声，有效地从成功和失败中学习，而无需承担权重更新带来的高计算成本或灾难性遗忘风险。在实验方面，我们在四个多样化基准上验证了 MemRL，包括 HLE、BigCodeBench、ALFWorld 和 Lifelong Agent Bench。我们的结果展示了相对于基线的一致优越性，在探索密集型环境中实现了相对改进。深入分析揭示了学习到的效用值与任务成功之间的强相关性，进一步证实了 MemRL 的有效性。总结来说，我们的贡献有三方面：

- 我们提出了一个基于模型-记忆解耦和意图-经验-效用三元组的运行时学习框架，以调和稳定性-可塑性困境，实现无需调参的智能体学习。
- 我们引入了 MemRL，一种通过两阶段检索和效用驱动更新实现智能体自我进化的非参数方法。
- 我们进行了广泛的评估，并为 MemRL 的稳定性提供了严格分析，展示了它如何确保任务完整性并最小化遗忘。

引言核心要点

- **灵感来源**：人类大脑的”建构性情景模拟”——通过回忆和重组过去经验来解决新问题
- **现有方法的缺陷**：微调太贵且会遗忘；RAG 只看相似不看效用
- **MemRL 三大支柱**：(1) Intent-Experience-Utility 三元组记忆结构；(2) 两阶段检索（先召回后排序）；(3) 基于环境反馈的 Q 值更新
- **核心哲学**：冻结大脑 (LLM)，升级笔记本 (外部记忆)

3 相关工作 (Related Works)

3.1 运行时学习 (Runtime Learning)

原文完整翻译

运行时学习关注智能体通过交互流进行部署后改进，而非依赖离线数据，标志着向”经验时代”的转变【Silver 2025——**David Silver 提出的”经验时代”理念，认为未来 AI 将主要从自生成的交互数据中学习，本文的核心动机之一**】。与通常更新参数来处理遗忘或分布偏移的持续学习【Parisi 2019, Wu 2024——**持续学习综述**】或测试时自适应【Sun 2020, Wang 2020 (TENT), Liang 2025——**测试时适应方法，在推理时更新模型参数**】不同，我们的设定约束骨干模型保持冻结以确保稳定性和效率。虽然最近的记忆增强智能体【Zheng 2025 (Lifelong Agent Bench)、Wei 2025 (Evo)、Zhou 2025 (RuleArena)——**利用外部记忆进行部署后学习的最新工作**】强调记忆组织，但选择问题——在反馈下识

别哪些经验应被重用——仍是一个关键挑战。借鉴价值感知的情景控制【**Tulving 1972——情景记忆概念的提出者；McClelland 1995——互补学习系统理论**】，我们将运行时学习框架化为识别有价值的片段。通过使用交互反馈来分配效用，我们的方法在不修改权重的情况下引导检索和重用，从而确保持续改进【**Pritzel 2017——Neural Episodic Control，将情景记忆引入 RL**】。

通俗解释

运行时学习的核心问题是：AI 部署之后，还能不能像人一样“越用越聪明”？

现有方法分为三类：（1）**持续学习**——不断更新模型参数，但容易忘掉旧知识；（2）**测试时自适应**——推理时临时调整参数，但不稳定；（3）**记忆增强**——存储经验供未来使用，但不知道存的经验有没有用。

MemRL 的定位是：**不碰模型参数**（保持稳定），而是在外部记忆上做强化学习来识别“哪些经验最值得重用”。就像一个有经验的医生，他的医学知识（大脑）不会轻易改变，但他的临床笔记（记忆）会不断更新——标注哪些治疗方案在哪些情况下有效。

3.2 强化学习 (Reinforcement Learning)

原文完整翻译

强化学习已被广泛用于增强 LLM。一个代表性范式是从人类反馈构建奖励信号，并据此优化模型策略以与人类偏好对齐【**Stiennon 2020 (RLHF)、Ouyang 2022 (InstructGPT)——用 RL 对齐 LLM 的开创性工作**】。其他近期方法利用基于规则的验证器来改善 LLM 的推理能力【**Guo 2025 (DeepSeek-R1)、Yu 2025 (DAPO)——使用规则验证器进行 RL 训练的最新工作**】。同时，面向智能体的研究探索了交互信号如何改善工具使用和动作决策【**Schick 2023 (Toolformer)——教 LLM 学习使用工具**】。尽管奖励驱动的优化展示了有效性，但这些方法通常将学习置于模型参数或额外的参数化模块中，因此无法避免在线更新的成本或遗忘的风险。相比之下，我们的方法将记忆使用框架化为可学习的决策问题，并在记忆上应用**非参数**强化学习来规避这一风险。

3.3 智能体记忆 (Agentic Memory)

原文完整翻译

为了避免微调的成本，外部记忆系统已从静态 RAG 范式发展为动态、可治理的记忆结构【**Lewis 2020 (RAG), Karpukhin 2020 (DPR)——RAG 的基础工作**】。早期智能体记忆引入了反思机制和层级管理来处理长上下文经验【**Shinn 2023 (Reflexion)——通过语言反思进行强化学习的工作；Packer 2024 (MemGPT)——将 LLM 作为操作系统管理记忆的工作**】。更近期的框架系统化了记忆生命周期，关注统一存储和用于复杂任务的结构化索引【**Li 2025 (MemOS)——将记忆抽象为可治理资源的框架；Xu 2025 (A-MEM)——强调网络链接和记忆进化；Huang 2025 (LiCoMemory)、Ye 2025 (Task Memory Engine)——改进长视野一致性的工作**】。此外，自适应方法现在探索通过反馈驱动的更新或自动增强来改进检索【**Salama 2025 (MemInsight)、Zhang 2025 (Learn to Memorize)、Li 2025 (RFM-RAG)、Zhou 2025 (Memento)——改进记忆检索的最新工作**】。然而，除了训练额外的可学习模块外，大多数现有方法仍主要依赖语义相似度或启发式规则，缺乏严格的指标来评估记忆在最大化回报方面的实际效用。受记忆再巩固的认知理论启发【**Schacter 2007, Gick 1980, Nader 2000——关于记忆再巩固的认知科学研究，说明记忆在检索后会被重新评估和更新**】，MemRL 通过将检索形式化为基于价值的决策过程来弥合这一差距，从环境奖励中学习稳健的效用估计（Q 值）以区分高价值经验。

相关工作定位总结

MemRL 在相关工作中的独特定位：

- 相比持续学习：不更新参数，通过外部记忆实现可塑性
- 相比RLHF/RL微调：RL 作用于记忆空间而非模型参数空间
- 相比RAG：不仅按相似度检索，还按学习到的效用值排序
- 相比智能体记忆：引入了严格的 RL 理论框架（Q-learning）来评估和更新记忆效用

4 问题形式化 (Problem Formulation)

原文完整翻译

在本节中，我们正式定义了记忆增强生成的问题，并建立了智能体策略与记忆检索之间的理论联系。我们采用记忆马尔可夫决策过程（M-MDP）的形式化 **【Zhou 2025 (Memento) ——提出 M-MDP 框架的工作，MemRL 基于此进行构建】**，并在其上应用我们的非参数强化学习方法。图 2 提供了这一记忆增强决策过程的示意性例子，展示了检索结果和记忆进化如何在多个时间步中展开。

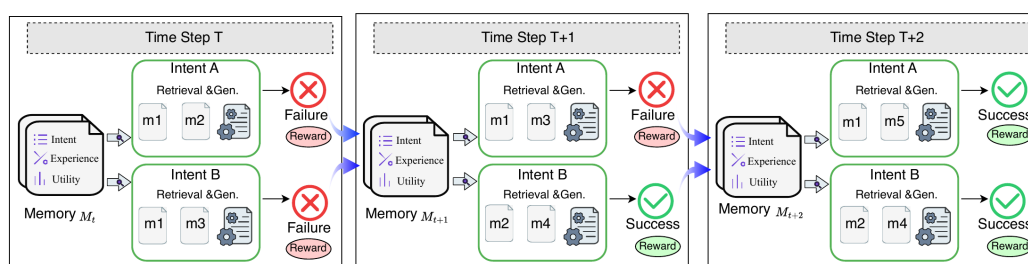


Figure 2: 记忆增强马尔可夫决策过程示例。在时间步 t ，智能体从初始记忆集 $\mathcal{M}_t$ 开始。在时间步 $t+1$ ，意图 A 检索到相关经验但生成失败；意图 B 成功，其经验被加入记忆。在时间步 $t+2$ ，意图 A 检索到意图 B 新存储的成功经验，导致成功结果。

图 2 详细解读

这张图展示了记忆如何在不同任务之间实现知识共享：

时间步 t ：智能体拥有一个初始的记忆库 $\mathcal{M}_t$ 。

时间步 $t+1$ ：两个不同的任务同时进行。”意图 A”（比如”如何配置 Nginx”）检索到了一些记忆但最终失败了。”意图 B”（比如”如何部署 Web 服务”）成功了，其成功经验被写入记忆库。

时间步 $t+2$ ：当”意图 A”再次出现时，它现在可以检索到之前”意图 B”存储的成功经验（因为配置 Nginx 和部署 Web 服务有相似的操作模式），从而成功完成任务。

核心洞察：这展示了 MemRL 的跨任务知识迁移能力——一个任务学到的经验可以帮助解决另一个相似的任务，即使它们不是完全相同的问题。

4.1 记忆增强智能体策略 (Memory-Augmented Agent Policy)

原文完整翻译

为使智能体能够自我进化，我们采用 M-MDP 框架 **[Zhou 2025 (Memento)—M-MDP 框架的来源]**，定义为元组 $(\mathcal{S}, \mathcal{A}, P, \mathcal{R}, \gamma, \mathcal{M})$ 。其中， $\mathcal{S}$ 和 $\mathcal{A}$ 分别表示状态空间和动作空间， P 是转移动态， $\mathcal{R}$ 是关于状态和动作的奖励函数， $\gamma \in [0, 1)$ 表示折扣因子， $\mathcal{M} = (\mathcal{S} \times \mathcal{A} \times \mathbb{R})^*$ 构成包含过去经验的进化记忆库。在每个时间步 t ，智能体接收状态 s_t 并利用 $\mathcal{M}_t$ 生成最大化预期奖励的响应 a_t 。联合策略 $\pi(a_t|s_t, \mathcal{M}_t)$ 被形式化为对所有可能检索项 m 的边缘概率：

$$\pi(a_t|s_t, \mathcal{M}_t) = \sum_{m \in \mathcal{M}_t} \mu(m|s_t, \mathcal{M}_t) p_{LLM}(a_t|s_t, m) \quad (1)$$

其中 $\mu(m|s_t, \mathcal{M}_t)$ 是用于选择记忆上下文的**检索策略**， $p_{LLM}(a_t|s_t, m)$ 是由冻结 LLM 参数化的**推理策略**。这种方法将检索从被动匹配转变为主动决策过程，有效地考虑了 m 在生成成功结果 a_t 中的功能效用。

在以前的 RAG 或基于记忆的智能体范式中，检索策略 μ 通常由固定的向量相似度度量（如嵌入的余弦相似度）确定。虽然对语义匹配有效，但这种策略无法考虑记忆的效用——即检索 m 是否实际导致了成功的结果 a_t 。

公式 1 数学推导详解

联合策略分解：

这个公式表达的是一个**全概率公式**的思想。智能体生成动作 a_t 的过程分两步：

第一步——检索：从记忆库 $\mathcal{M}_t$ 中选择一条记忆 m ，概率为 $\mu(m|s_t, \mathcal{M}_t)$ 。

第二步——生成：基于当前状态 s_t 和检索到的记忆 m ，由冻结的 LLM 生成动作 a_t ，概率为 $p_{LLM}(a_t|s_t, m)$ 。

合并：总的生成概率 = 对所有可能的记忆 m 求和（边缘化），即：

$$\pi(a_t|s_t, \mathcal{M}_t) = \sum_{m \in \mathcal{M}_t} \underbrace{\mu(m|s_t, \mathcal{M}_t)}_{\text{选择记忆 } m \text{ 的概率}} \cdot \underbrace{p_{LLM}(a_t|s_t, m)}_{\text{给定 } m \text{ 后生成 } a_t \text{ 的概率}}$$

符号说明：

- $s_t \in \mathcal{S}$ ：当前状态（如用户的查询或任务描述），维度为嵌入向量维度（如 3072 维）
- $a_t \in \mathcal{A}$ ：智能体的输出动作（如生成的代码、执行的操作序列）
- m ：从记忆库中检索到的一条记忆条目
- $\mathcal{M}_t$ ：时间步 t 时的记忆库，包含所有已存储的经验
- μ ：检索策略——这是 MemRL 要优化的对象
- p_{LLM} ：冻结的 LLM 生成策略——这个不更新

数值例子：假设记忆库中有 3 条记忆 $\{m_1, m_2, m_3\}$ ，检索概率分别为 $\mu = (0.6, 0.3, 0.1)$ ，LLM 在各记忆下生成正确答案的概率为 $p_{LLM} = (0.8, 0.2, 0.5)$ ，则总成功概率为： $0.6 \times 0.8 + 0.3 \times 0.2 + 0.1 \times 0.5 = 0.48 + 0.06 + 0.05 = 0.59$ 。

4.2 非参数强化学习 (Non-Parametric Reinforcement Learning)

原文完整翻译

为了克服静态相似度的局限性，我们通过将记忆检索形式化为基于价值的决策过程来操作化 M-MDP 框架 **【Zhou 2025 (Memento)——M-MDP 操作化的参考】**。与通过权重更新优化 π_{LLM} 的参数化方法不同，我们通过将 M-MDP 组件映射到结构化的意图-经验-效用三元组，在记忆空间内直接优化检索策略 $\mu(m|s, \mathcal{M})$ 。

从语义匹配到决策制定。我们将状态 s 实例化为用户意图，由当前查询的嵌入向量封装 **【Lewis 2020——使用嵌入进行检索的方法】**。因此，动作空间 $\mathcal{A}_t$ 变得动态且离散，对应于从当前记忆库 $\mathcal{M}_t$ 中选择特定的 m **【Zhou 2025 (Memento)】**。在这一形式化中，检索不再是被动的匹配任务，而是为增强生成器的动作 a 而采取的策略性决策步骤。

通过 Q 值定义效用。虽然智能体的可执行动作 a 由策略 π 生成（如第 4.1 节所形式化），但这一生成的质量严格取决于检索到的上下文。因此，我们将传统的价值函数 $Q(s, a)$ 适配到检索阶段，定义 $Q(s, m)$ 为由记忆 m 增强的后续动作 a 的预期效用。MemRL 学习一个最优检索策略 μ^* 来最大化此效用：

$$\mu^*(m|s, \mathcal{M}) = \arg \max_{m \in \mathcal{M}} Q(s, m) \quad (2)$$

在这个视角下，Q 值充当检索机制的评论家 (Critic)，将策略性地辅助生成器的记忆与仅具有高语义相似度的无关噪声区分开来。

非参数学习。由于检索动作空间与 LLM 生成解耦，我们可以在不修改模型权重的情况下进行学习。在收到环境反馈 r 后，我们通过时间差分 (TD) 误差更新 Q 值 **【Sutton 1988——TD 学习的原始论文】**：

$$Q(s, m) \leftarrow Q(s, m) + \alpha[r + \gamma \max_{m'} Q(s', m') - Q(s, m)] \quad (3)$$

或蒙特卡洛风格规则 **【Metropolis 1949——蒙特卡洛方法的原始论文】**：

$$Q_{\text{new}} \leftarrow Q_{\text{old}} + \alpha(r - Q_{\text{old}}) \quad (4)$$

其中 α 是学习率。公式 4 是公式 3 通过将 s' 设为终止状态而自然简化的版本，以平衡复杂性和性能，与 DeepSeek-R1 共享类似的单步 MDP 形式化 **【Guo 2025 (DeepSeek-R1)——使用类似单步 MDP 的最新工作】**。这些方式使效用估计随时间收敛到真实的预期回报 **【Bellman 1966——动态规划的奠基人】**。通过在记忆结构内显式更新 Q 值，MemRL 提供了一种具有理论保证的非参数学习方式，使智能体能够通过交互实现自我进化。

公式 3 和 4 数学推导详解

完整的 Q-Learning 更新（公式 3）：

这是经典的 Bellman 最优方程的随机近似版本。其含义是：

$$Q(s, m) \leftarrow \underbrace{Q(s, m)}_{\text{旧估计}} + \underbrace{\alpha}_{\text{学习率}} \left[\underbrace{r}_{\text{即时奖励}} + \underbrace{\gamma \max_{m'} Q(s', m')}_{\text{未来最优回报的折扣估计}} - \underbrace{Q(s, m)}_{\text{旧估计}} \right]$$

简化的蒙特卡洛更新（公式 4）：

当我们把每次任务交互视为一个**单步 MDP**（即任务完成后就到达终止状态，没有后续状态 s' ），那么 $\gamma \max_{m'} Q(s', m') = 0$ ，公式 3 简化为：

$$Q_{\text{new}} \leftarrow Q_{\text{old}} + \alpha(r - Q_{\text{old}})$$

这本质上是一个**指数移动平均 (EMA)** 更新。

数值例子：假设某条记忆当前 $Q_{\text{old}} = 0.3$ ，学习率 $\alpha = 0.3$ 。

情况1——使用该记忆后任务成功 ($r = 1$)：

$$Q_{\text{new}} = 0.3 + 0.3 \times (1 - 0.3) = 0.3 + 0.21 = 0.51$$

Q 值从 0.3 上升到 0.51——这条记忆”更有价值了”。

情况2——使用该记忆后任务失败 ($r = 0$)：

$$Q_{\text{new}} = 0.3 + 0.3 \times (0 - 0.3) = 0.3 - 0.09 = 0.21$$

Q 值从 0.3 下降到 0.21——这条记忆”价值降低了”。

关键直觉：经过多次更新后，Q 值会收敛到该记忆在类似任务中的**平均成功率**。如果一条记忆被使用了 100 次，其中 70 次成功，Q 值将趋近于 0.7。

注意事项

单步 MDP vs 完整 MDP：论文中实际使用的是简化的蒙特卡洛更新（公式 4），而非完整的 TD 更新（公式 3）。虽然完整的 TD 更新理论上更强大，但单步版本在实践中已经足够好——因为每次任务交互（如”解决一道编程题”）本身就可以被视为一个完整的 episode。这与 DeepSeek-R1 中使用 GRPO 时的类似简化思路一致。

5 MemRL 方法 (Method)

原文完整翻译

基于第 4 节定义的 M-MDP 形式化，我们提出 MemRL，一个使冻结 LLM 通过非参数强化学习实现自我进化的框架。MemRL 不修改模型权重 θ ，而是在进化的记忆空间内优化**检索策略** $\mu(m|s, \mathcal{M})$ 。如图 3 所示，该框架由三个核心组件组成：(i) 结构化的**意图-经验-效用**记忆库，(ii) 将语义召回与价值感知选择解耦的**两阶段检索机制**，(iii) 稳定 Q 值估计的**运行时效用更新规则**。

图 3 详细解读

这是 MemRL 的完整系统架构图，分为三个主要部分：

上方——端到端学习循环：这是 MemRL 的主流程。给定一个新的查询/任务 s （比如”写一个排序算法”），系统会：(1) 从记忆库 $\mathcal{M}$ 中检索相关经验 m_{ctx} ；(2) 将 s 和 m_{ctx} 一起送入冻结的 LLM 生成输出 y ；(3) 环境给出奖励 R （成功=1，失败=0）；(4) 用 R 更新被检索记忆的 Q 值。

左下——两阶段检索：Phase-A（语义召回）：用余弦相似度从全部记忆中找出前 k_1 个最相似的候选。Phase-B（价值排序）：用一个综合得分（结合相似度和 Q 值）从 k_1 个候选中选出最终的 k_2 个。这保证了检索结果既相关又有用。

右下——效用更新：任务完成后，根据成功/失败结果更新被使用记忆的 Q 值。成功的经验 Q 值升高（变得更容易被未来检索到），失败的经验 Q 值降低（变得不太容易被检索到）。同时，新的经验会被总结并写入记忆库。

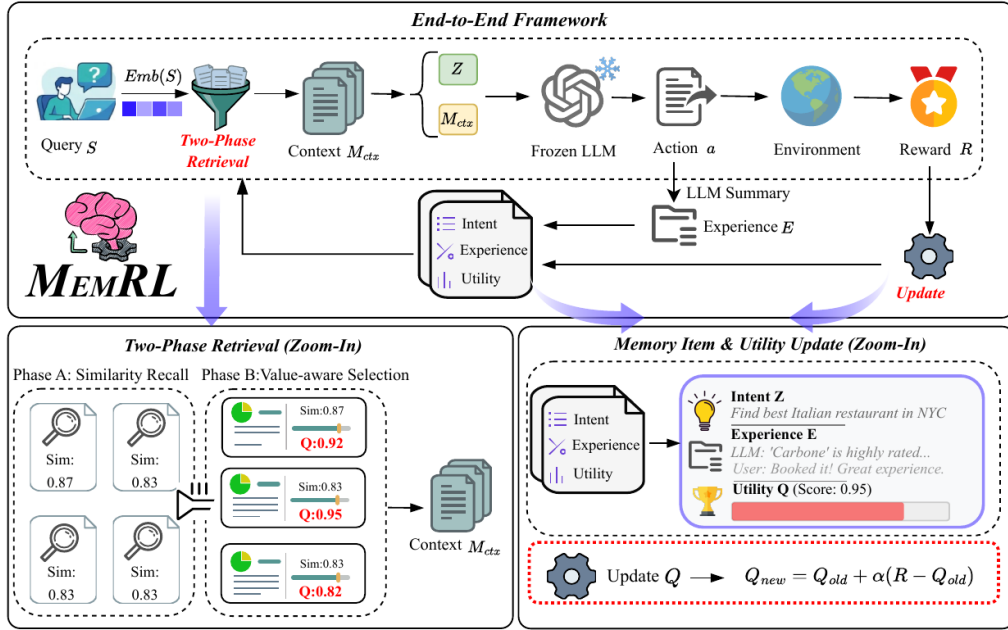


Figure 3: **MemRL 框架概览**。端到端学习循环（上）：给定查询 s ，智能体从记忆 M 中检索上下文 m_{ctx} ，生成输出 y ，并基于奖励 R 更新记忆价值 Q 。**两阶段检索**（左下）：候选记忆先通过相似度召回，再通过学习到的 Q 值重排序。**效用更新**（右下）：使用环境奖励更新 Q 值，以区分功能效用与语义相似度。

5.1 意图-经验-效用三元组 (The Intent-Experience-Utility Triplet)

原文完整翻译

为了支持基于价值的决策，我们将外部记忆 $\mathcal{M}$ 不仅仅作为键值对来结构化，而是作为一组三元组：

$$\mathcal{M} = \{(z_i, e_i, Q_i)\}_{i=1}^{|\mathcal{M}|} \quad (5)$$

其中 z_i 表示**意图** (Intent)， e_i 存储原始**经验** (Experience，如成功的解题轨迹)， Q_i 表示学习到的**效用** (Utility)。 Q_i 近似于将经验 e_i 应用于与 z_i 类似意图时的预期回报，充当 RL 中的评论家 (Critic)。

通俗解释

把记忆库想象成一本”经验笔记本”，每一页（每条记忆）包含三个字段：

意图 z_i （”遇到了什么问题？”）：将任务描述编码成向量，方便后续查找相似问题。比如”配置 Nginx 服务器”对应一个 3072 维的向量。

经验 e_i （”当时是怎么做的？”）：存储完整的解决方案或操作轨迹。比如”第1步：安装 nginx；第2步：编辑配置文件；...”

效用 Q_i （”这个做法靠谱吗？”）：一个 0 到 1 之间的数字，表示这条经验在过去类似任务中的平均成功率。 $Q_i = 0.8$ 意味着”用了这条经验，80% 的情况下能成功”。

这比传统 RAG 的”查询-文档”键值对多了一个关键维度——**效用**。这就像不仅记住”做过什么”，还记住”做得好不好”。

5.2 从语义召回到价值感知选择 (Two-Phase Retrieval)

原文完整翻译

标准 RAG 系统假设”相似即有用”，但智能体任务常涉及泛化能力差的环境特定例程【Singh 2025, Cuconasu 2024, Gan 2024, Zhou 2024——展示语义相似不一定有用的研究】。因此，MemRL 实现了两阶段检索策略。

Phase A：基于相似度的召回。给定查询 s ，我们通过余弦相似度和稀疏阈值 δ 过滤记忆库 $\mathcal{M}$ ，隔离出一个语义一致的候选池 $\mathcal{C}(s)$ ：

$$\mathcal{C}(s) = \text{TopK}_{k_1}(\{i \mid \text{sim}(\text{Emb}(s), \text{Emb}(z_i)) > \delta\}) \quad (6)$$

其中 Emb 表示将原始文本转换为向量的嵌入模型。如果 $\mathcal{C}(s) = \emptyset$ ，MemRL 仅依赖冻结的 LLM 进行探索。

Phase B：价值感知选择。为确定最终上下文 $\mathcal{M}_{ctx}(s)$ ，我们使用一个平衡探索（相似度）和利用（效用 Q ）的复合得分从 $\mathcal{C}(s)$ 中选择前 k_2 项：

$$\text{score}(s, z_i, e_i) = (1 - \lambda) \cdot \hat{\text{sim}}(\text{Emb}(s), \text{Emb}(z_i)) + \lambda \cdot \hat{Q}_i \quad (7)$$

其中 $\hat{\cdot}$ 表示 z-score 标准化， $\lambda \in [0, 1]$ 调节权衡。该机制过滤掉”干扰性”记忆——那些语义相似但历史效用低的记忆。如第 6.4.2 节所述，标准化和严格的相似度阈值对于噪声过滤和维持自我进化期间的稳定性至关重要。

公式 6 和 7 数学推导详解

Phase A——语义召回（公式 6）：

步骤1：计算查询 s 与每条记忆意图 z_i 的余弦相似度：

$$\text{sim}(\text{Emb}(s), \text{Emb}(z_i)) = \frac{\text{Emb}(s) \cdot \text{Emb}(z_i)}{\|\text{Emb}(s)\| \cdot \|\text{Emb}(z_i)\|}$$

步骤2：过滤掉相似度低于阈值 δ 的记忆（ δ 根据数据集自适应设定，取前 20% 分位数）。

步骤3：从剩余记忆中取前 k_1 个（按相似度降序）。

数值例子：假设记忆库有 100 条记忆， $\delta = 0.5$ ， $k_1 = 10$ 。计算后发现 30 条记忆的相似度 > 0.5 ，则从这 30 条中取相似度最高的 10 条作为候选池 $\mathcal{C}(s)$ 。

Phase B——价值排序（公式 7）：

步骤1：对候选池内的相似度和 Q 值分别做 z-score 标准化：

$$\hat{\text{sim}}_i = \frac{\text{sim}_i - \bar{\text{sim}}}{\sigma_{\text{sim}}}, \quad \hat{Q}_i = \frac{Q_i - \bar{Q}}{\sigma_Q}$$

步骤2：计算综合得分， $\lambda = 0.5$ 时为等权平衡：

$$\text{score}_i = 0.5 \times \hat{\text{sim}}_i + 0.5 \times \hat{Q}_i$$

步骤3：取得分最高的 k_2 个作为最终上下文。

数值例子：假设候选池中有两条记忆，标准化后的值为：

- 记忆 A: $\hat{\text{sim}} = 1.2$ ， $\hat{Q} = -0.5$ ，得分 $= 0.5 \times 1.2 + 0.5 \times (-0.5) = 0.35$
- 记忆 B: $\hat{\text{sim}} = 0.3$ ， $\hat{Q} = 1.5$ ，得分 $= 0.5 \times 0.3 + 0.5 \times 1.5 = 0.90$

虽然记忆 A 更相似，但记忆 B 的效用更高，最终 B 被优先选中。这就是”相似不一定有用”的精髓。

5.3 基于记忆的非参数强化学习 (Non-Parametric RL on Memory)

原文完整翻译

MemRL 的核心是基于环境反馈持续精化 Q 值，使智能体能够“记住”什么有效。在运行时，MemRL 完全在记忆空间中执行学习。利用检索到的上下文 m ，智能体根据公式 1 中定义的策略 π 采样一个动作 a 。执行 a 后产生环境奖励 r （如执行成功或标量分数）。对于实际注入输入上下文的记忆 $\mathcal{M}_{\text{ctx}}(s)$ ，我们使用蒙特卡洛风格规则（即公式 4）更新三元组中的效用值，遵循图 3 所示的运行时学习循环。

这一更新驱动 Q_{new} 趋向于在类似意图下使用经验 e_i 的经验预期回报。同时，对于每个采样的轨迹，我们使用 LLM 来总结经验 **【Fang 2025 (MemP)——使用 LLM 总结经验的方法】**，并将其作为新的三元组 $(z, e_{\text{new}}, Q_{\text{init}})$ 写回记忆库，实现经验的持续扩展同时保持 LLM 参数不变。

MemRL 方法核心总结

MemRL 的完整运行循环如下：

1. **接收任务**：智能体获得新任务 s （如“写一个快速排序函数”）
2. **Phase-A 召回**：用语义相似度找到 k_1 个候选记忆
3. **Phase-B 选择**：用综合得分（相似度 + Q 值）选出 k_2 个最终记忆
4. **生成动作**：将任务 + 记忆送入冻结 LLM，生成解决方案
5. **获取反馈**：环境给出奖励（成功=1，失败=0）
6. **更新 Q 值**：用公式 4 更新被使用记忆的效用值
7. **存储新经验**：将本次经验总结后写入记忆库
8. 回到步骤1，处理下一个任务

5.4 理论稳定性分析 (Theoretical Stability Analysis)

原文完整翻译

我们从强化学习角度分析 MemRL 的稳定性，完整分析见附录 A。我们提出两个标准假设：冻结的推理策略 p_{LLM} 和平稳的任务分布。在这些条件下，学习目标 $\beta(s, m) = \mathbb{E}[r_t | s, m]$ 是良定义的，其中期望取自推理策略 p_{LLM} 分布产生的随机奖励。我们证明通过公式 4 更新的效用估计是无偏的且方差有界的 **【Sutton & Barto 2018——RL 收敛理论】**。具体地，当 $t \rightarrow \infty$ 时：

$$\begin{aligned} \lim_{t \rightarrow \infty} \mathbb{E}[Q_t] &= \beta(s, m), \\ \limsup_{t \rightarrow \infty} \text{Var}(Q_t) &\leq \frac{\alpha}{2 - \alpha} \text{Var}(r_t | s, m). \end{aligned} \quad (8)$$

此外，我们通过将 MemRL 框架化为广义期望最大化（GEM）**【Dempster 1977——EM 算法的原始论文】** 过程来解决训练期间潜在检索分布 $\text{Pr}(s|m)$ 偏移的挑战。系统在全局目标上执行坐标上升：检索排序充当策略改进（E-step），效用更新充当值更新（M-step）。根据单调改进定理 **【Neal & Hinton 1998——GEM 单调改进定理】**，系统收敛到一个稳定点，全局记忆效用稳定：

$$\lim_{t \rightarrow \infty} \mathbb{E}[Q_t(m)] = \sum_{s \in \mathcal{S}(m)} \mathbb{E}[r | s, m] \text{Pr}(s|m) \quad (9)$$

其中 $\mathcal{S}(m)$ 是记忆的有效支持集。这一形式化保证了全局稳定性并防止灾难性遗忘。

稳定性分析数学推导详解

1. 期望收敛（公式 8 第一行）：

定义误差 $e_t = Q_t - \beta(s, m)$ ，根据更新规则：

$$e_{t+1} = (1 - \alpha)e_t + \alpha(r_t - \beta(s, m))$$

取条件期望（利用 $\mathbb{E}[r_t|s, m] = \beta(s, m)$ ）：

$$\mathbb{E}[e_{t+1}] = (1 - \alpha)\mathbb{E}[e_t]$$

迭代得： $\mathbb{E}[e_t] = (1 - \alpha)^t \mathbb{E}[e_0]$

由于 $0 < \alpha < 1$ ， $(1 - \alpha)^t \rightarrow 0$ ，所以 $\mathbb{E}[Q_t] \rightarrow \beta(s, m)$ 。

数值例子： $\alpha = 0.3$ ， $Q_0 = 0$ ， $\beta = 0.7$ 。

经过 $t = 1$ ： $\mathbb{E}[e_1] = 0.7 \times (-0.7) = -0.49$ ，即 $\mathbb{E}[Q_1] = 0.21$

经过 $t = 5$ ： $\mathbb{E}[e_5] = 0.7^5 \times (-0.7) = -0.118$ ，即 $\mathbb{E}[Q_5] \approx 0.58$

经过 $t = 10$ ： $\mathbb{E}[e_{10}] = 0.7^{10} \times (-0.7) \approx -0.020$ ，即 $\mathbb{E}[Q_{10}] \approx 0.68$

可见 Q 值快速收敛到真实值 0.7。

2. 方差有界（公式 8 第二行）：

方差递推： $v_{t+1} = (1 - \alpha)^2 v_t + \alpha^2 \sigma^2$

极限： $\lim_{t \rightarrow \infty} v_t = \frac{\alpha^2 \sigma^2}{\alpha(2 - \alpha)} = \frac{\alpha}{2 - \alpha} \sigma^2$

数值例子： $\alpha = 0.3$ ， $\sigma^2 = 0.25$ （对应成功率 0.5 的二元奖励）：

$$\text{Var}_{\infty}(Q) \leq \frac{0.3}{2 - 0.3} \times 0.25 = \frac{0.3}{1.7} \times 0.25 \approx 0.044$$

即标准差约为 0.21， Q 值不会剧烈振荡。

6 实验 (Experiments)

6.1 实验设置 (Experimental Setup)

原文完整翻译

基线与基准。我们将 MemRL 与 RAG 类（RAG、Self-RAG）、智能体记忆（Mem0、MemP）和测试时扩展（Pass@ k ）基线在冻结骨干设置下进行比较（详见附录 D）。评估涵盖四个领域：BigCodeBench（编码）、ALFWorld（导航）、Lifelong Agent Bench（OS/DB）和 Humanity’s Last Exam（HLE）。骨干模型根据基准选择以避免无信号或天花板问题，确保有效的学习信号，同时附录提供了统一比较以展示在相同容量下的跨任务一致性。

评估指标。我们采用两个指标：(1) **成功率 (SR)**，一个 epoch 中完成任务的比例；(2) **累积成功率 (CSR)**，跨 epoch 至少成功一次的任务比例。

我们在两个不同设置下评估 MemRL 和基线：**运行时学习**，评估在训练会话中学习和适应的能力；和**迁移**，评估学习到的记忆在未见任务上的泛化能力。

Table 1: 运行时学习结果。我们在 10 个 epoch 上比较 MemRL 与各种基线。结果报告为最后 epoch 成功率 / 累积成功率（CSR）。Average 列表示所有基准的平均性能。

Method	BigCodeBench	Lifelong Agent Bench		ALFWorld	HLE	Average
	Code Gen	OS Task	DB Task	Exploration	Knowledge	(Last / CSR)
Model	GPT-4o	GPT-4o-mini	GPT-4o-mini	GPT-5-mini	Gemini-3-pro	-
No Memory	0.485	0.674	0.860	0.777	0.357	0.631
Pass@10	- / 0.577	- / 0.756	- / 0.928	- / 0.928	- / 0.524	- / 0.743
RAG	0.475 / 0.483	0.690 / 0.700	0.914 / 0.916	0.887 / 0.930	0.430 / 0.475	0.679 / 0.699
Self-RAG	0.497 / 0.561	0.646 / 0.732	0.891 / 0.898	0.907 / 0.962	0.423 / 0.475	0.673 / 0.726
Mem0	0.487 / 0.495	0.670 / 0.702	0.920 / 0.926	0.894 / 0.969	0.436 / 0.470	0.681 / 0.712
MemP	0.578 / 0.602	0.736 / 0.742	0.960 / 0.966	0.885 / 0.919	0.522 / 0.570	0.736 / 0.760
MemRL	0.595 / 0.627	0.788 / 0.804	0.960 / 0.972	0.949 / 0.981	0.570 / 0.606	0.772 / 0.798

6.2 主要结果 (Main Results)

6.2.1 运行时学习结果

原文完整翻译

如表 1 所示，MemRL 在所有领域展示了稳健的优越性，在累积成功率（CSR）上平均超过最强基线（MemP）+3.8%。在探索密集型环境如 ALFWorld 和 OS 任务上增益最显著（均为 +6.2%），同时在具有挑战性的 HLE 基准上保持稳定领先（+3.6%）。这证实了我们的基于价值的机制与 MemP 的启发式检索不同，能有效过滤噪声以保留高效用的程序模式。

表 1 详细解读

这张表是论文的核心结果表，展示了 10 个 epoch 的运行时学习结果：
表格结构：每个单元格包含两个数字，”最后 epoch SR / CSR”。例如 MemRL 在 ALFWorld 上的 ”0.949 / 0.981” 意味着第 10 个 epoch 的成功率为 94.9%，整个训练过程中累计解决了 98.1% 的任务。
关键发现：

- MemRL 在**所有基准**上都取得了最佳或并列最佳结果
- ALFWorld（导航任务）上优势最大：SR 从 MemP 的 0.885 提升到 0.949（+7.2%），CSR 从 0.919 提升到 0.981（+6.2%）
- 即使在难度极高的 HLE 基准上也有显著提升（+4.8% SR， +3.6% CSR）
- 标准 RAG 有时甚至不如 No Memory（如 BigCodeBench），说明检索噪声记忆可能有害

各模型选择说明：不同基准使用不同骨干模型是为了避免”无信号”（模型太弱，几乎全部失败）或”天花板效应”（模型太强，几乎全部成功），确保记忆机制有足够的改进空间。

Table 2: **迁移学习结果**。在 BigCodeBench、Lifelong Agent Bench 和 ALFWorld 上的迁移结果。使用最佳验证结果。Average 列表示所有基准的平均成功率。

	BigCodeBench	Lifelong Agent Bench		ALFWorld	Average
Method	Code Gen	OS Task	DB Task	Exploration	SR
Model	GPT-4o	GPT-4o-mini	GPT-4o-mini	GPT-5-mini	-
No Memory	0.485	0.673	0.841	0.836	0.709
RAG	0.479	0.713	0.920	0.950	0.765
Self-RAG	0.500	0.653	0.881	0.950	0.746
Mem0	0.485	0.686	0.935	0.950	0.764
MemP	0.494	0.720	0.928	0.921	0.766
MemRL	0.508	0.746	0.942	0.979	0.794

6.2.2 迁移学习结果

原文完整翻译

我们通过在训练后冻结记忆库并在留出测试集上测试来评估记忆的可迁移性。如表 2 所示，MemRL 展现了优越的可迁移性，在成功率上平均超过最强基线（MemP）+2.8%。优势在复杂环境如 ALFWorld（+5.8%）和 OS 任务（+2.6%）上尤为明显。这些差距验证了我们的**两阶段检索**有效过滤了低价值噪声，保留了能稳健泛化到未见场景的高效程序模式。

6.3 消融实验 (Ablations)

6.3.1 运行时 RL 的有效性

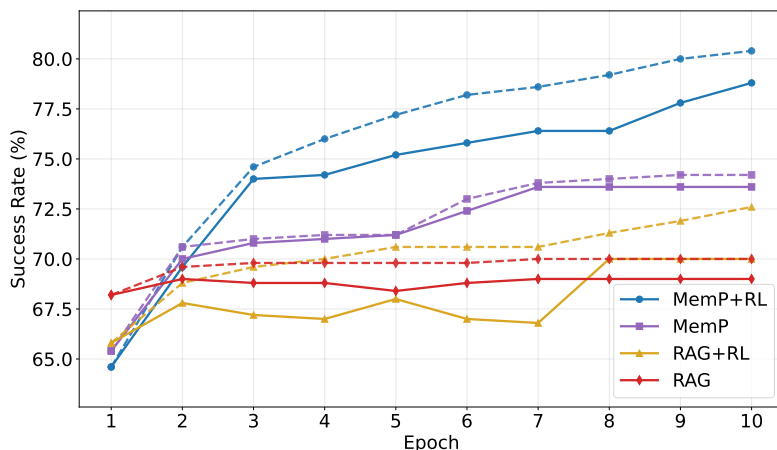


Figure 4: **OS 交互性能**。MemRL 与基线（MemP、RAG）的性能比较。**实线**表示 epoch 成功率，**虚线**表示累积成功率（CSR）。MemRL 展示了更好的稳定性和不断扩大的 CSR 差距。

图 4 详细解读

这张图展示了 OS 交互任务上各方法随 epoch 变化的性能：

实线（Epoch 成功率）：每个 epoch 的即时成功率。MemRL（蓝色实线）在后期 epoch 显著高于 MemP 和 RAG，且波动更小（更稳定）。

虚线（CSR）：至少成功过一次的任务累积比例。MemRL 的 CSR 虚线始终高于其他方

法，且差距单调递增——说明 RL 驱动的价值函数在持续帮助智能体“解锁”新的任务解法。

关键发现：初始阶段各方法差距不大（因为 Q 值还没有收敛），但随着训练推进，MemRL 的优势越来越明显。这正是因为 Q 值在多次更新后变得越来越准确，能更好地识别真正有用的记忆。

6.3.2 Q 值权重因子的影响

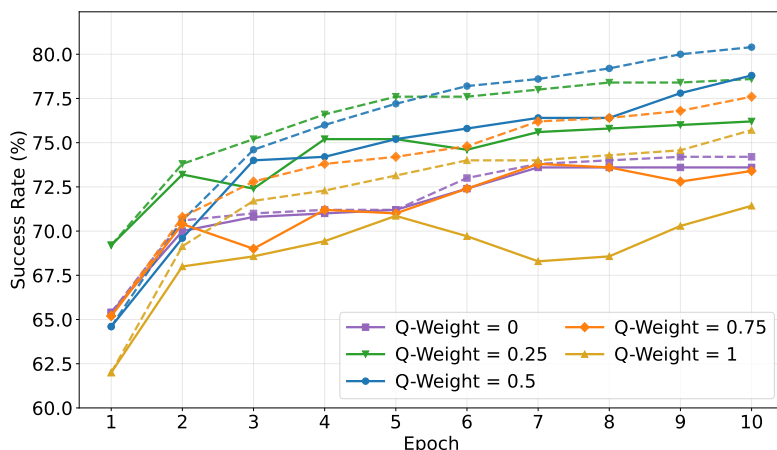


Figure 5: Q 值权重因子 λ 的消融。跨 $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ 的性能比较。实线表示 epoch 成功率，虚线表示 CSR。平衡设定 ($\lambda = 0.5$) 在相关性和有用性之间实现了最优权衡。

图 5 详细解读

这张消融图揭示了 λ （控制相似度与 Q 值权重平衡）的最优设定：

$\lambda = 0$ （纯语义检索）：完全不使用 Q 值，退化为标准 RAG。性能平稳但缺乏改进动力。

$\lambda = 0.5$ （平衡设定）：取得最佳效果。语义相似度保证检索的“相关性”，Q 值保证检索的“有用性”。

$\lambda = 1$ （纯 RL 检索）：完全忽略语义相似度，只看 Q 值。出现严重的不稳定和性能下降。这是因为发生了“上下文脱节”——一条记忆在某个领域 Q 值很高，但被用到了毫不相关的任务上。

核心结论：语义相似度是必要的“锚点”，Q 值是在此基础上的“优化梯度”。两者缺一不可。

6.3.3 检索范围消融：跨任务 vs 单任务

Table 3: 检索范围消融。比较 MemRL 与限制为单任务反思的消融版本。

Setting	BCB	OS	DB	ALFWorld	HLE	Avg.
Single-Task Reflection	0.614	0.714	0.938	0.930	0.610	0.761
MemRL (Cross-Task)	0.627	0.804	0.972	0.981	0.606	0.798

表 3 解读

这个消融研究将 MemRL 的跨任务记忆检索与单任务反思（类似 Reflexion）进行对比：

核心发现：MemRL 在 OS (+9.0%) 和 ALFWorld (+5.1%) 上大幅领先，因为这些环境中任务之间高度相似——一个任务学到的操作模式可以直接帮助解决另一个相似任务（水平迁移）。

有趣例外：HLE 上单任务反思 (0.610) 略高于 MemRL (0.606)。这是因为 HLE 的任务间语义相似度很低（仅 0.186），跨任务迁移效果有限，此时“专注于自己的反思”反而更有效。

6.3.4 检索规模敏感性分析

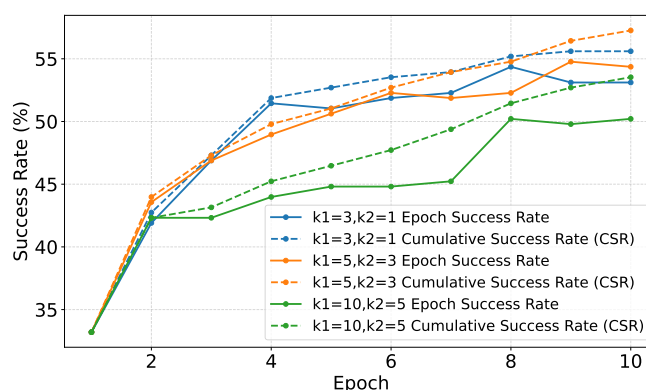


Figure 6: 检索规模 (k_1, k_2) 消融。在 HLE (CS/AI) 子集上稀疏 (3/1)、中等 (5/3) 和密集 (10/5) 检索设置的性能比较。中等设置实现了最优权衡。

图 6 详细解读

这张图展示了检索数量对性能的影响，呈现出明显的倒U型趋势：

稀疏设置 ($k_1 = 3, k_2 = 1$)：检索太少，信息不足，性能受限。

中等设置 ($k_1 = 5, k_2 = 3$)：最佳平衡点，检索到足够的有用信息但不引入太多噪声。

密集设置 ($k_1 = 10, k_2 = 5$)：检索太多记忆反而导致性能下降——过多的上下文可能“淹没”关键信息，干扰 LLM 的推理。

核心结论：记忆的质量比数量更重要。MemRL 的价值在于精选少量高效用记忆，而非堆砌大量相似但未经筛选的内容。

6.4 讨论 (Discussion)

6.4.1 Q 评论家的预测能力

图 7 详细解读

图 (a) 成功率 vs Q 值：将记忆按 Q 值分成不同区间，统计每个区间内记忆被使用时的实际成功率。结果显示：

- 最低 Q 值区间：实际成功率仅 21.5%
- 最高 Q 值区间：实际成功率达 88.1%
- Pearson 相关系数 $r = 0.861$ （非常强的正相关）

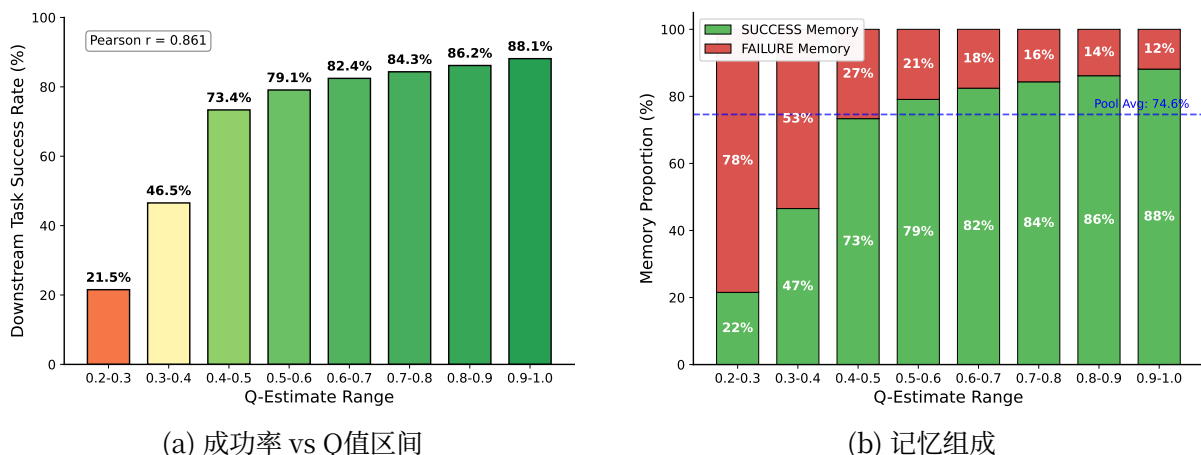


Figure 7: **Q 值分析**。(a) Pearson $r = 0.861$ 证实了评论家的预测能力。(b) 高 Q 值区间中约 12% 的失败记忆表明了潜在的策略效用。

这证明 Q 值确实学到了有意义的效用估计，能准确预测记忆的实际有用性。

图 (b) 记忆组成分析：即使在高 Q 值 (0.9–1.0) 的区间内，仍有约 12% 的记忆来源于“失败”的经历。这不是 Q 值的错误——这些“高价值失败记忆”往往是近似成功 (near-miss) 的经验，包含了可迁移的策略教训。例如，一次失败的操作中“发现空输出不等于错误”的反思，在后续任务中可以帮助避免同类错误。

6.4.2 MemRL 的稳定性

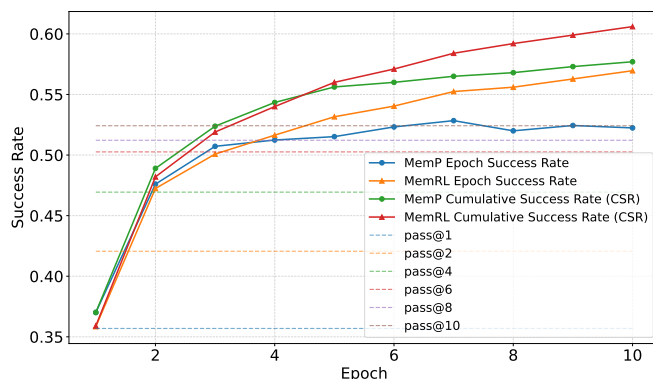


Figure 8: HLE 中 MemRL 和 MemP 的 epoch 成功率与累积成功率 (CSR)。

图 8 详细解读

这张图揭示了 MemRL 相对于 MemP 的**稳定性优势**：

MemP (蓝色)：CSR (虚线) 持续上升，但 epoch SR (实线) 波动较大，且两者之间的**间隙越来越大**。这个间隙代表“灾难性遗忘”——智能体虽然曾经解决过某些任务，但后来检索到了错误的记忆导致无法重现成功。

MemRL (红色)：CSR 和 epoch SR**同步增长**，间隙很小。这意味着 MemRL 不仅能发现新的解决方案，还能稳定地保持已有的成功——这正是理论稳定性分析 (GEM 收敛保证) 在实践中的体现。

遗忘率数据：MemRL 的平均遗忘率仅为 0.041，远低于 MemP 的 0.051。去除 z-score 标准化和相似度门控后，遗忘率飙升至 0.073，证明这些设计对稳定性至关重要。

7 局限与结论 (Limitations and Conclusion)

原文完整翻译

局限与未来工作。虽然 MemRL 为非参数进化奠定了基础，但其运行时动态揭示了几个有前景的方向。(i) 当前的步进式更新虽然快速，但在长视野轨迹中可能引入高方差噪声，启发我们探索多步更新或定期记忆整合。(ii) 效用更新期间的信用分配模糊性，特别是当多条经验被引用时，需要更精确的归因方法，如 Shapley 方法【Shapley 1953——合作博弈论中的 Shapley 值】或多智能体强化学习中的值分解【Rashid 2020 (QMIX), Sunehag 2017 (VDN)——值分解方法】。(iii) 虽然 MemRL 随任务暴露增加而改善，但当任务相似度低时，性能可能退化为类反思行为，突显了维护充足且多样化经验库的重要性。

结论。我们提出了 MemRL，一种通过将记忆检索视为基于价值的决策过程来调和稳定性-可塑性困境的非参数方法。通过意图-经验-效用三元组结构和蒙特卡洛风格更新，MemRL 使智能体能够自我进化并区分高效用策略与语义噪声，而无需权重更新。广泛的评估证实 MemRL 在运行时适应和泛化方面显著优于基线。在静态训练数据变得稀缺的未来，智能体在其生命周期中通过交互产生的经验将成为新的、至关重要的知识来源。我们希望这一范式为构建稳定的、持续学习的智能体铺平道路，使其能从交互中高效适应。

8 影响声明 (Impact Statement)

原文完整翻译

本文提出了 MemRL，一种旨在增强 LLM 智能体长期推理能力的价值强化记忆检索机制。从更广泛的角度来看，我们的工作有助于开发更高效、更可靠的自主系统。通过优化记忆检索过程，MemRL 减少了大规模智能体部署的计算开销，可能降低 AI 基础设施对环境的影响。此外，随着 LLM 智能体越来越多地融入日常工作流程，对稳健记忆机制的研究有助于确保这些系统在其行动中保持根基和一致性。我们预见这一算法进步不会带来直接的负面社会后果，尽管我们承认所有自主系统的部署都应配备适当的人类监督，以减轻与 AI 决策相关的更广泛风险。

A 附录 A：理论分析与证明

A.1 稳定性分析

原文完整翻译

我们从强化学习角度分析 MemRL 的稳定性，关注记忆中存储的效用估计的收敛行为。
设置。在每个时间步 t ，智能体观察意图状态 s_t ，检索记忆项 $m_t \in \mathcal{M}_t$ ，生成输出 a_t ，并接收标量奖励 $r_t \in [-1, 1]$ 。

平稳奖励假设。我们在固定数据集上分析学习过程，并提出两个关键条件：(1) 冻结推理策略；(2) 固定任务分布。这些假设保证学习目标是良定义的：

$$\mathbb{E}[r_t | s_t = s, m_t = m] = \beta(s, m) \quad (10)$$

定理（EMA 估计收敛）。令 $\{Q_t\}$ 按公式 4 以恒定步长 $\alpha \in (0, 1]$ 更新。若公式 10 成立且状态-记忆对 (s, m) 被无限次更新，则：

$$\lim_{t \rightarrow \infty} \mathbb{E}[Q_t] = \beta(s, m) \quad (11)$$

收敛速率为指数级： $\mathbb{E}[Q_t] - \beta(s, m) = (1 - \alpha)^t (Q_0 - \beta(s, m))$ 。

定理证明详解

证明过程：

定义估计误差 $e_t \triangleq Q_t - \beta(s, m)$ 。

第1步——误差递推关系。将更新规则 $Q_{t+1} = (1 - \alpha)Q_t + \alpha r_t$ 代入：

$$\begin{aligned} e_{t+1} &= Q_{t+1} - \beta = (1 - \alpha)Q_t + \alpha r_t - \beta \\ &= (1 - \alpha)(e_t + \beta) + \alpha r_t - \beta \\ &= (1 - \alpha)e_t + \alpha(r_t - \beta) \end{aligned}$$

第2步——取条件期望。利用 $\mathbb{E}[r_t | s, m] = \beta$ ：

$$\mathbb{E}[e_{t+1} | \mathcal{F}_t] = (1 - \alpha)e_t + \alpha(\beta - \beta) = (1 - \alpha)e_t$$

第3步——取全期望并迭代：

$$\mathbb{E}[e_t] = (1 - \alpha)^t \mathbb{E}[e_0] = (1 - \alpha)^t (Q_0 - \beta)$$

由于 $0 < \alpha < 1$ ， $(1 - \alpha)^t \rightarrow 0$ ，故 $\mathbb{E}[Q_t] \rightarrow \beta$ 。□

A.2 有界方差

原文完整翻译

若奖励方差 $\text{Var}(r_t | s, m) = \sigma^2 < \infty$ ，则 Q_t 的方差保持有界。

方差递推： $v_{t+1} = (1 - \alpha)^2 v_t + \alpha^2 \sigma^2$

递归展开得： $v_t = (1 - \alpha)^{2t} v_0 + \alpha^2 \sigma^2 \sum_{k=0}^{t-1} ((1 - \alpha)^2)^k$

渐近收敛：

$$\limsup_{t \rightarrow \infty} \text{Var}(Q_t) = \frac{\alpha^2 \sigma^2}{\alpha(2 - \alpha)} = \frac{\alpha}{2 - \alpha} \sigma^2$$

因此，恒定步长更新不会引起无界振荡，而是产生稳定的效用估计。

A.3 通过变分推断的收敛性（GEM 分析）

原文完整翻译

全局变分目标 $\mathcal{J}(\mu, Q)$:

$$\mathcal{J}(\mu, Q) = \mathbb{E}_{s \sim \mathcal{D}} \left[\sum_{m \in \mathcal{S}(s)} \mu(m|s) Q(s, m) - \frac{1}{\beta} D_{\text{KL}} \left(\mu(\cdot|s) \parallel \pi_{\text{sim}}(\cdot|s) \right) \right] \quad (12)$$

E-step（策略优化）：固定 Q ，优化 μ 。通过 Lagrange 乘子法求解得到最优策略为 Boltzmann 分布：

$$\mu^*(m|s) \propto \pi_{\text{sim}}(m|s) \exp(\beta Q(s, m))$$

取对数恢复 Phase-B 得分函数： $\log \mu^*(m|s) \propto \log \pi_{\text{sim}}(m|s) + \beta Q(s, m)$

M-step（值更新）：固定 μ ，最小化 MSE： $\min_Q \mathbb{E}[(r - Q(s, m))^2]$ ，对应公式 4 的 SGD 步。

根据 GEM 单调改进定理，交替优化保证系统收敛到稳定点，检索策略稳定，从而保证 Q 值收敛到真实预期回报。

B 附录 B：扩展分析

B.1 遗忘动态分析

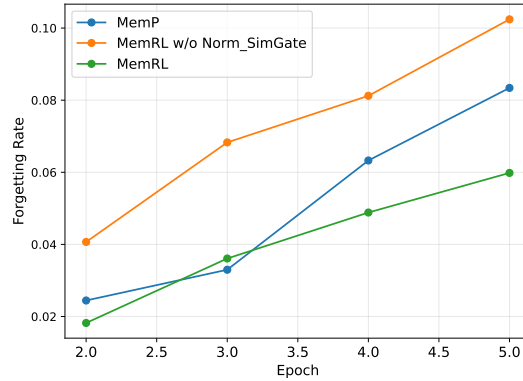


Figure 9: HLE 上的遗忘率轨迹。比较 MemRL、MemP 和消融版本（无标准化和门控）的累积遗忘率。曲线越低表示稳定性越好。

图 9 详细解读

此图展示了三种配置的遗忘率随训练推进的变化：

MemRL（蓝色）：遗忘率始终保持在最低水平（均值 0.041），说明 RL 驱动的检索策略在学习新知识的同时能很好地保留旧知识。

MemP（橙色）：遗忘率逐渐上升（均值 0.051），说明启发式检索方法会逐渐“覆盖”之前有效的策略。

无标准化和门控的消融（绿色）：遗忘率大幅飙升（均值 0.073），并出现明显的尖峰。这证明 z-score 标准化和相似度门控是防止“噪声策略”扰乱已有知识的关键设计。

Table 4: 任务结构的影响。CSR 增益比较。多步任务从 MemRL 中获益显著更多。

Benchmark	交互类型	MemP (%)	MemRL (%)	增益 (pp)
ALFWorld	多步	91.9	98.1	+6.2
OS Task	多步	74.2	80.4	+6.2
HLE	单步	57.0	60.6	+3.6
BigCodeBench	单步	60.2	62.7	+2.5

B.2 MemRL 作为轨迹验证器

表 4 解读

这个表揭示了一个重要发现：MemRL 在**多步序列任务**上的增益远大于单步任务。这是因为在多步任务中，检索到的记忆必须对**整条轨迹**都有效。标准语义检索可能找到初始步骤匹配的记忆，但这些记忆在后续步骤中可能完全失效。MemRL 通过将最终奖励反向传播到记忆效用 Q ，学会了验证**整条轨迹的结构完整性**，因此充当了”轨迹验证器”的角色。

B.3 任务相似度对记忆效力的影响

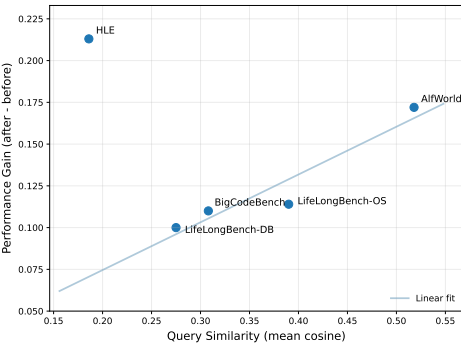


Figure 10: 基于相似度的泛化分析。数据集内语义相似度与 MemRL 性能增益之间的关系。

图 10 详细解读

此图分析了数据集内部任务相似度与 MemRL 性能提升之间的关系：

正相关趋势：总体而言，数据集内任务相似度越高，MemRL 的性能提升越大（线性回归趋势线斜率为正）。ALFWorld（相似度 0.518）获得了最大的增益。

HLE 异常：HLE 相似度最低（0.186），但性能增益却很高（+0.213）。这因为 HLE 的增益来自**运行时记忆化**——通过重复暴露来记住特定问题的解法，而非跨任务泛化。

启示：MemRL 支持两种学习模式——高相似度环境中的**模式泛化**和低相似度环境中的**特定知识获取**。

Table 5: 跨模型迁移结果（HLE）。各智能体使用原始由 Gemini-3-pro 训练的冻结记忆库的性能。 Δ 为绝对提升。

推理模型	基础分数	迁移分数	增益 (Δ)
Qwen3-235B	0.150	0.531	+0.381
Gemini-3-flash	0.347	0.583	+0.236
GPT-5.2(High)	0.354	0.571	+0.217

C 附录 C：高级能力——迁移与合并

C.1 跨模型记忆迁移

表 5 解读

这个实验极具说服力：用 Gemini-3-pro 训练的记忆库，**直接**（无需任何调整）迁移到其他完全不同的模型上，都能获得巨大提升。特别是 Qwen3-235B 从 0.150 飙升到 0.531（超过 3 倍提升）。

这说明 MemRL 学到的是**模型无关**的问题解决模式——如高效的代码骨架和推理模板——而非特定模型的产物。记忆库可以作为**便携式知识库**，让弱模型”继承”强模型的能力。

C.2 多任务记忆合并

Table 6: 记忆合并结果。使用任务特定独立记忆库与合并统一记忆库的性能比较。

基准任务	独立记忆	合并记忆	Δ
Lifelong-OS	0.788	0.784	-0.004
Lifelong-DB	0.960	0.960	0.000

表 6 解读

将 OS 和 DB 两个领域的记忆库直接合并使用，性能几乎无损失（OS 仅降 0.4%，DB 完全不变）。这得益于两阶段检索中 Phase-A 的语义过滤——OS 命令和 SQL 查询的语义空间本身就正交，无关的跨任务记忆在第一阶段就被自动排除。这意味着 MemRL 支持**模块化记忆组合**，可以通过简单合并记忆模块来扩展能力。

D 附录 D：实现细节

D.1 模型配置

D.2 超参数设置

超参数设置解读

几个关键超参数值得注意：

学习率 $\alpha = 0.3$ ：所有基准统一使用，说明该超参数不敏感，无需逐任务调优。

$\lambda = 0.5$ ：相似度和 Q 值等权平衡，如消融实验所验证。

相似度阈值 δ ：这是唯一需要根据数据集调整的超参数。ALFWorld 的阈值最高（0.62），因为其任务间相似度本身就很高，需要更严格的过滤。HLE 阈值最低（0.25），因为其任

Table 7: 模型与 API 配置。

组件	配置/版本	备注
骨干 LLM	GPT-4o	BigCodeBench
	GPT-4o-mini	Lifelong Bench
	GPT-5-mini	ALFWorld
	Gemini-3-pro	HLE
嵌入模型	Text-Embedding-3-Large	意图和查询编码
生成参数	Temperature = 0.0	通用（贪婪解码）
	Temperature = 0.6	HLE 专用

Table 8: 各基准的超参数设置。

参数	描述	BCB	OS	DB	ALF	HLE
α	学习率	0.3	0.3	0.3	0.3	0.3
λ	Q 权重平衡	0.5	0.5	0.5	0.5	0.5
δ	相似度阈值	0.38	0.50	0.37	0.62	0.25
k_1	Phase-A 召回量	10	10	10	5	5
k_2	Phase-B 选择量	5	5	5	3	3
Q_{init}	初始 Q 值	0.0	0.0	0.0	0.0	0.0

务多样性大，过高的阈值会排除几乎所有记忆。

D.3 数据划分

Table 9: 数据划分与数据集大小摘要。

基准	运行时学习	迁移学习	划分说明
HLE	2,500 tasks	–	全集
Lifelong-OS	500 tasks	500 tasks	7:3 划分
Lifelong-DB	500 tasks	500 tasks	7:3 划分
BigCodeBench	1,140 tasks	1,140 tasks	7:3 划分
ALFWorld	3,553 tasks	140 tasks	valid_seen

E 附录 E：统一骨干对比实验

表 10 解读

使用统一的 GPT-4o-mini 骨干，MemRL 在所有基准上都一致性地优于基线。特别是在 ALFWorld 上，MemRL 的 CSR 从 MemP 的 0.413 大幅提升到 0.680（+26.7%），证明即使骨干模型较弱，MemRL 的记忆优化机制也能显著增强性能。

Table 10: GPT-4o-mini 跨基准的运行时学习性能

Method	OS		DB		ALFWorld	
	SR	CSR	SR	CSR	SR	CSR
No Memory	0.674	–	0.860	–	0.278	–
Pass@10	–	0.756	–	0.928	–	0.462
MemP	0.736	0.742	0.960	0.966	0.299	0.413
MemRL	0.788	0.804	0.960	0.972	0.440	0.680

F 附录 F：效率分析

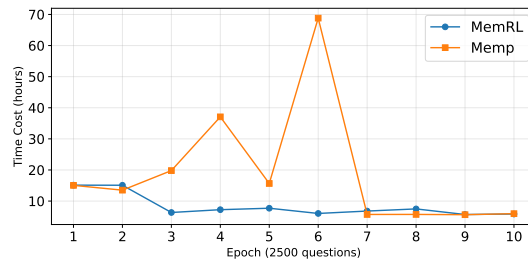


Figure 11: HLE 上每 epoch 的挂钟时间。MemRL 展示了稳定的运行时间，确认算法开销可忽略。

图 11 解读

此图展示 MemRL 与 MemP 在实际运行时间上几乎无差异。MemRL 的额外计算（双阶段检索中的向量点积和 Q 值标量更新）都是毫秒级操作，相比 LLM 生成 2500 个问题所需的数小时来说完全可以忽略。MemP 曲线上的尖峰（如 Epoch 4、6）来自 API 网络延迟波动，与算法复杂度无关。

G 附录 G：参考文献一览表

以下表格整理了论文中引用的主要文献，包含其核心内容和在本文中的引用原因。

Table 11: 参考文献一览表

文献	主要内容	本文引用原因
Grossberg 2013	自适应共振理论，研究大脑中的稳定性-可塑性平衡	提供认知科学中稳定性-可塑性困境的理论基础
McClelland 1995	互补学习系统理论	说明人脑通过海马体（快速学习）和新皮层（慢速整合）的分离来避免灾难性遗忘
Schacter 2007	建构性情景模拟理论	MemRL的核心灵感来源——人类通过重组过去经验来模拟未来场景
Sutton & Barto 2018	强化学习经典教材	提供Q-learning等RL算法的理论基础
Lewis 2020 (RAG)	检索增强生成框架	MemRL 构建于 RAG 之上，但增加了效用评估维度
Shinn 2023 (Reflexion)	基于语言反思的RL智能体	MemRL的消融对比对象之一（单任务反思）
Fang 2025 (MemP)	程序化记忆框架	MemRL的主要基线对比方法和记忆写入机制的来源
Zhou 2025 (Memento)	提出M-MDP框架	MemRL 直接采用 其 M-MDP形式化作为理论基础
Guo 2025 (DeepSeek-R1)	使用RL提升LLM推理	共享类似的单步MDP简化思路
Silver 2025	” 经验时代” 理念	MemRL的核心动机——智能体应从自生成的交互经验中学习
Kirkpatrick 2017 (EWC)	弹性权重整合防止灾难性遗忘	说明参数更新方法的遗忘风险
Watkins 1992	Q-learning原始论文	MemRL的Q值更新理论来源
Dempster 1977	EM算法	MemRL 收敛性分析中 GEM框架的来源
Neal & Hinton 1998	GEM单调改进定理	保证MemRL的E-step/M-step交替优化收敛
Bellman 1966	动态规划	Q值更新的理论根基

文献	主要内容	本文引用原因
Ouyang 2022	InstructGPT (RLHF)	说明传统RL用于LLM对齐的范式
Rafailov 2023 (DPO)	直接偏好优化	参数化RL对齐方法的代表, 与非参数MemRL对比
Packer 2024 (MemGPT)	LLM作为操作系统管理记忆	智能体记忆系统的代表性工作
Zheng 2025 (LLB)	Lifelong Agent Bench	本文使用的评测基准之一
Zhuo 2025 (BCB)	BigCodeBench	本文使用的评测基准之一
Shridhar 2021 (ALF-World)	文本世界导航基准	本文使用的评测基准之一
Phan 2025 (HLE)	Humanity's Last Exam	本文使用的评测基准之一